Prompt para Claude Code — Agente WhatsApp Local

> Copia TODO el bloque de abajo y pégalo a Claude Code en una carpeta
> vacía donde ejecutes `claude`. Funciona end-to-end con todas las
> lecciones aprendidas ya incorporadas.

```
# CONTEXTO DEL PROYECTO

Vas a construir un agente de WhatsApp local que se conecta a un número
real vía Baileys (no Meta API, no Twilio) y responde mensajes con un
LLM. Incluye un dashboard local para ver las conversaciones, leer el
historial, intervenir manualmente y togglear cada chat entre modo IA
(responde el bot) y modo Humano (responde la persona desde el
dashboard).

Todo corre en localhost. La data vive en SQLite (archivo local). La
sesión de WhatsApp Web la guarda Baileys en una carpeta local.

# OBJETIVO FINAL

Cuando termines, debe funcionar esto:

1. `npm run start:bot` levanta Baileys. Si NO hay sesión guardada en
   `./auth/`, queda esperando a que el usuario escanee el QR desde
   el frontend (NO desde la terminal — el QR principal vive en el
   dashboard). En la terminal también se imprime el QR ASCII como
   fallback de debugging, pero el usuario final escanea desde el
   navegador.
2. `npm run dev` levanta el dashboard Next.js en localhost:3000.
   Cuando el usuario abre la página por primera vez:
   - Si NO hay sesión Baileys conectada, el dashboard muestra una
     pantalla "Conectar número" con el QR renderizado como imagen
     PNG en grande.
   - Cuando Baileys detecta la conexión exitosa, la pantalla
     transiciona automáticamente al dashboard real (lista de
     conversaciones + panel) sin que el usuario tenga que recargar.
   - Header del dashboard muestra el número conectado y un botón
     "Desconectar" que borra la sesión y vuelve a la pantalla de QR.
3. Después del escaneo, Baileys guarda la sesión en `./auth/`. En
   reinicios posteriores del proceso bot, NO se vuelve a pedir QR
   mientras la sesión siga viva en WhatsApp.
4. Cuando alguien escribe al WhatsApp del usuario:
   - guardar el mensaje en SQLite,
   - si la conversación está en modo "AI", llamar a OpenRouter con el
     historial reciente y el system prompt, guardar la respuesta y
```

enviarla por Baileys de vuelta;
  - si la conversación está en modo "HUMAN", solo guardar y NO responder.
5. Dashboard real (después de conectar):
  - lista de conversaciones a la izquierda (ordenadas por último mensaje, más reciente arriba);
  - panel de conversación a la derecha (mensajes user/bot/human con timestamp);
  - toggle AI/HUMAN por chat (arriba del panel derecho);
  - input de texto + botón "Enviar" cuando el chat está en HUMAN (envía un mensaje firmado como "human" desde el dashboard, llega al cliente vía Baileys);
  - botón "Borrar" en el panel para borrar una conversación con diálogo de confirmación;
  - polling cada 2 segundos a un endpoint que devuelve mensajes nuevos y el estado de la conexión (NO usar WebSocket en v1).

STACK OBLIGATORIO

- Next.js 16 App Router + TypeScript + React 19. Turbopack default.
- Tailwind CSS 4
- @whiskeysockets/baileys 6.7+ — cliente WhatsApp Web vía QR
- better-sqlite3 11+ — base de datos local (file-based)
- pino — logger requerido por Baileys (level: silent)
- qrcode — genera el QR como Data URL (PNG base64) en el server
- qrcode-terminal — fallback ASCII en la consola del bot
- openai SDK (apuntando a OpenRouter via baseURL)
- tsx — para ejecutar scripts TS directamente
- concurrently — para levantar bot + Next.js juntos en producción
- Node.js 20+ (Baileys, Next.js 16, Tailwind 4 lo requieren)

NO usar Prisma, Drizzle, Supabase, Redis, WebSockets, Vercel, Meta API oficial ni Twilio.

ESTRUCTURA DE CARPETAS

```
agente-whatsapp/
├── src/
│   ├── app/
│   │   ├── page.tsx              # renderiza ConnectionGate
│   │   ├── layout.tsx
│   │   ├── globals.css
│   │   └── api/
│   │       ├── connection/
│   │       │   ├── status/route.ts   # GET estado + QR PNG
│   │       │   └── disconnect/route.ts
│   │       ├── conversations/
```

```
│   │       │   ├── route.ts          # GET lista
│   │       │   └── [conversationId]/route.ts  # DELETE
│   │       ├── messages/
│   │       │   └── [conversationId]/route.ts  # GET + POST
│   │       └── mode/
│   │           └── [conversationId]/route.ts  # POST cambia AI/HUMAN
│   ├── components/
│   │   ├── ConnectionGate.tsx
│   │   ├── QRScreen.tsx
│   │   ├── DashboardHeader.tsx
│   │   ├── ConversationList.tsx
│   │   ├── ConversationPanel.tsx
│   │   ├── MessageBubble.tsx
│   │   └── ModeToggle.tsx
│   └── lib/
│       ├── db.ts
│       ├── openrouter.ts
│       ├── system-prompt.ts
│       └── baileys/
│           ├── client.ts
│           └── handler.ts
├── scripts/
│   ├── env-loader.ts              # CRÍTICO: side-effect import
│   └── start-bot.ts
├── data/                          # gitignored, runtime
├── auth/                          # gitignored, sesión Baileys
├── .env.local
├── .env.example
├── .gitignore
├── package.json
├── tsconfig.json
├── next.config.ts
├── postcss.config.mjs
├── Procfile                       # para deploy con buildpack
├── nixpacks.toml                  # para EasyPanel/Railway
├── .nvmrc                         # para forzar Node 22
└── README.md
```

VARIABLES DE ENTORNO

`.env.example`:
```
OPENROUTER_API_KEY=sk-or-...
OPENROUTER_MODEL=openai/gpt-4o-mini
```

Recomienda al usuario `openai/gpt-4o-mini` — los modelos `:free` de

OpenRouter tienen rate limits muy estrictos (50 requests/día sin créditos cargados) y van a fallar en producción real con error 429.

PACKAGE.JSON

Campo `engines` OBLIGATORIO:
```json
"engines": {
  "node": ">=20.9.0"
}
```

Scripts:
```json
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "next lint",
  "start:bot": "tsx scripts/start-bot.ts",
  "start:all": "concurrently --kill-others --names BOT,WEB --prefix-colors yellow,cyan \"npm run start:bot\" \"npm run start\""
}
```

`tsx` y `concurrently` van en `dependencies` (no devDependencies), porque si no fallan en producción cuando el buildpack ejecuta `npm ci --omit=dev`.

SCHEMA SQLITE

`src/lib/db.ts` debe inicializar better-sqlite3 apuntando a `./data/messages.db`, crear la carpeta y archivo si no existen, y ejecutar este DDL al arrancar (incluye PRAGMA WAL para concurrencia entre el proceso bot y el de Next.js):

```sql
PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS conversations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  phone TEXT UNIQUE NOT NULL,
  name TEXT,
  mode TEXT CHECK(mode IN ('AI','HUMAN')) NOT NULL DEFAULT 'AI',
  last_message_at INTEGER,
  created_at INTEGER NOT NULL DEFAULT (unixepoch())
);
```

```
CREATE TABLE IF NOT EXISTS messages (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  conversation_id INTEGER NOT NULL REFERENCES conversations(id),
  role TEXT CHECK(role IN ('user','assistant','human')) NOT NULL,
  content TEXT NOT NULL,
  created_at INTEGER NOT NULL DEFAULT (unixepoch())
);

CREATE INDEX IF NOT EXISTS idx_messages_conv
  ON messages(conversation_id, created_at);

CREATE TABLE IF NOT EXISTS connection_state (
  id INTEGER PRIMARY KEY CHECK (id = 1),
  status TEXT CHECK(status IN ('disconnected','qr','connecting','connected'))
    NOT NULL DEFAULT 'disconnected',
  qr_string TEXT,
  phone TEXT,
  updated_at INTEGER NOT NULL DEFAULT (unixepoch())
);

INSERT OR IGNORE INTO connection_state (id, status) VALUES (1, 'disconnected');

CREATE TABLE IF NOT EXISTS outbox (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  conversation_id INTEGER NOT NULL,
  phone TEXT NOT NULL,
  content TEXT NOT NULL,
  sent INTEGER NOT NULL DEFAULT 0,
  created_at INTEGER NOT NULL DEFAULT (unixepoch())
);

CREATE INDEX IF NOT EXISTS idx_outbox_pending
  ON outbox(sent, created_at);
```

`connection_state` es una fila única que sirve de "buzón" entre
el proceso bot y el de Next.js (corren en procesos separados).
`outbox` también — los mensajes humanos del dashboard se encolan
ahí, el bot los lee cada 2s y los envía vía Baileys.

Helpers que `db.ts` debe exportar:
- `getOrCreateConversation(phone, name?)` — `{ id, phone, name, mode, ... }`
- `getConversationById(id)` — `Conversation | null`
- `insertMessage(conversationId, role, content)` — TRANSACCIONAL:
  insert + UPDATE last_message_at en la misma transacción
- `getMessages(conversationId, limit = 50)`
- `getRecentHistory(conversationId, limit = 20)` — devuelve los

últimos N en orden cronológico ASCENDENTE (consulta DESC + reverse en JS, mucho más eficiente que ORDER BY ASC sobre toda la tabla)
- `setMode(conversationId, mode)`
- `listConversations()` — incluye `last_message_preview` con subquery para evitar N+1
- `getConnectionState()` y `setConnectionState({status, qr_string?, phone?})` — `setConnectionState` PRESERVA campos no provistos: si pasas solo `{status: 'connecting'}` el `qr_string` previo NO se borra. Solo pasar `null` explícito borra. Esto importa para no perder el QR durante transiciones intermedias.
- `enqueueOutbox(conversationId, phone, content)`
- `getPendingOutbox(limit = 20)`
- `markOutboxSent(id)`
- `deleteConversation(id)` — borra mensajes + outbox pendiente + conversación en una transacción atómica. Outbox `sent=1` se deja como histórico (NO se borra).

⚠️ LECCIÓN APRENDIDA — CARGA DE .env.local EN start-bot.ts

`scripts/start-bot.ts` es un proceso separado de Next.js. Necesita leer `.env.local` manualmente. El bug clásico:

```typescript
// ❌ MAL — los `import` se hoistean al top y corren ANTES del loadEnv()
function loadEnv() { ... }
loadEnv();
import { generateReply } from "../src/lib/openrouter"; // ya leyó undefined
```

ES modules hoistean TODOS los imports al inicio del archivo, sin importar dónde los escribiste. Si `openrouter.ts` lee `process.env.OPENROUTER_API_KEY` en su top-level, va a leer `undefined` porque el loadEnv() todavía no se ejecutó.

Solución: poner el loader en su propio módulo e importarlo PRIMERO:

```typescript
// scripts/env-loader.ts — solo side effects, sin exports
import path from "node:path";
import fs from "node:fs";
const envPath = path.resolve(process.cwd(), ".env.local");
if (fs.existsSync(envPath)) {
  const text = fs.readFileSync(envPath, "utf-8");
  for (const rawLine of text.split(/\r?\n/)) {
    const line = rawLine.trim();
    if (!line || line.startsWith("#")) continue;
    const eq = line.indexOf("=");
    if (eq < 0) continue;
```

```
    const key = line.slice(0, eq).trim();
    let value = line.slice(eq + 1).trim();
    if ((value.startsWith("'") && value.endsWith("'")) ||
        (value.startsWith("""") && value.endsWith"""))) {
      value = value.slice(1, -1);
    }
    if (!(key in process.env)) process.env[key] = value;
  }
}
```

```typescript
// scripts/start-bot.ts — env-loader DEBE ser el primer import
import "./env-loader";
import path from "node:path";
import fs from "node:fs";
import { ... } from "../src/lib/db";
// ...
```

Este orden funciona porque los imports siguen orden de declaración
DENTRO del bloque hoisted. Como `env-loader` no tiene exports,
solo ejecuta side effects (poblar process.env).

⚠️ LECCIONES APRENDIDAS — CONFIGURACIÓN DE BAILEYS

Estas decisiones son CRÍTICAS. Sin ellas el bot entra en loops o
no conecta:

1. fetchLatestBaileysVersion() OBLIGATORIO

WhatsApp rechaza versiones desactualizadas con error code 405.
Baileys hardcodea una versión que queda vieja entre releases.
SIEMPRE descarga la más nueva en runtime:

```typescript
import { fetchLatestBaileysVersion } from "@whiskeysockets/baileys";

let version: [number, number, number] | undefined;
try {
  const fetched = await fetchLatestBaileysVersion();
  version = fetched.version;
} catch (err) {
  console.warn("[bot] No se pudo obtener última versión:", err);
}

const sock = makeWASocket({ version, ... });
```

2. Browser fingerprint conocido (NO custom)

Si pasas un `browser: ['Mi App', 'Chrome', '1.0']` custom, WhatsApp trata la sesión como dispositivo desconocido y dispara code 440 (connectionReplaced) en loop apenas conectas. Usa SIEMPRE `Browsers.macOS('Desktop')`:

```typescript
import { Browsers } from "@whiskeysockets/baileys";

const sock = makeWASocket({
  version,
  auth: state,
  logger,
  browser: Browsers.macOS("Desktop"),  // ← crítico
  markOnlineOnConnect: false,
  syncFullHistory: false,
});
```

3. NO uses `printQRInTerminal: true`

Está deprecated en Baileys 6.7+. Va a tirar warning y eventualmente falla. Maneja el QR tú: escucha el evento `connection.update`, recibes el `qr` raw string, haces lo que quieras (DB + ASCII con qrcode-terminal).

4. State machine del connection.update

Reglas estrictas para evitar bugs sutiles:

- Cuando llega `qr` (string): `setConnectionState({status: 'qr', qr_string: qr, phone: null})`.
- Cuando `connection === 'connecting'`: SOLO setear `'connecting'` si el estado actual es `'disconnected'` (primer arranque). NO degradar desde `'qr'` ni desde `'connected'`.
- Cuando `connection === 'open'`: setear `'connected'` con el phone extraído de `sock.user.id` (formato `5491155...:N@s.whatsapp.net`, partir por `:` y tomar la parte numérica).
- Cuando `connection === 'close'`:
  - Si el status code es `DisconnectReason.loggedOut` (401): setear `'disconnected'`, borrar qr_string y phone, NO reconectar.
  - Cualquier otro code: NO modificar el estado de la DB. Solo schedule un reconnect. La razón: si estás `'connected'`, quieres seguir mostrando "connected" en el dashboard mientras el bot reconecta transparentemente. Si la reconexión necesita un nuevo QR, el evento `qr` va a sobreescribir el estado.

5. Backoff específico para code 440

Code 440 = `connectionReplaced`. Ocurre típicamente justo después del pairing inicial: WhatsApp abre un WS "definitivo" mientras el de pairing está activo, y kickea uno. Si reintentas muy rápido (3s) entras en loop. Espera 15s para code 440, 5s para los demás:

```typescript
const delay = code === 440 ? 15000 : 5000;
```

6. Cleanup del socket viejo antes de reconectar

```typescript
function scheduleReconnect(code) {
  if (reconnectTimer) return;
  reconnectTimer = setTimeout(() => {
    reconnectTimer = null;
    if (handle) {
      try { handle.sock.end(undefined); } catch {}
      handle = null;
    }
    start();
  }, delay);
}
```

Sin el `sock.end()`, Baileys puede dejar listeners colgando que se mezclan con la nueva conexión.

⚠️ LECCIÓN APRENDIDA — API ROUTE STATUS

El endpoint `GET /api/connection/status` debe devolver el QR PNG si `qr_string` existe AUNQUE el status no sea exactamente `'qr'`. Esto es defensivo: por race conditions a veces el bot tiene qr_string seteado pero status='connecting'. Si la API solo mira status, el frontend nunca ve el QR.

```typescript
const shouldShowQr =
  !!state.qr_string &&
  (state.status === "qr" || state.status === "connecting");
if (shouldShowQr && state.qr_string) {
  const qrPng = await QRCode.toDataURL(state.qr_string, { width: 320, margin: 2 });
  return NextResponse.json({ status: "qr", qrPng, updatedAt: state.updated_at });
}
```

⚠️ NEXT.JS 16 — params es Promise

Todos los route handlers con `[segmento]` dinámico:

```typescript
// ✅ Next.js 16
interface Ctx { params: Promise<{ conversationId: string }>; }
export async function GET(_req: NextRequest, { params }: Ctx) {
  const { conversationId } = await params;
}

// ❌ Sintaxis Next 14 — falla en build
export async function GET(_req, { params }: { params: { conversationId: string } }) {
  const { conversationId } = params;
}
```

Lo mismo con `cookies()` y `headers()` de `next/headers` — son async.

⚠️ NEXT.JS — serverExternalPackages

Sin esto, Next.js intenta empaquetar baileys/better-sqlite3/pino
en su bundle del server y rompe. `next.config.ts`:

```typescript
import type { NextConfig } from "next";
const nextConfig: NextConfig = {
  serverExternalPackages: ["@whiskeysockets/baileys", "better-sqlite3", "pino"],
};
export default nextConfig;
```

OUTBOX — SEPARACIÓN DE PROCESOS

Como bot y Next.js corren en procesos distintos, no comparten memoria.
La API no puede llamar `sock.sendMessage()` directamente. El flujo
para mensajes humanos:

1. POST `/api/messages/[id]` con role 'human':
   - INSERT en `messages` con role='human' (visible en el dashboard inmediatamente)
   - INSERT en `outbox` con phone + content + sent=0
   - Devuelve `{ ok: true, messageId }`

2. El proceso bot tiene un `setInterval` cada 2s que:
   - `SELECT * FROM outbox WHERE sent=0`
   - Por cada uno: `sock.sendMessage(jid, { text: content })`
   - Si OK: `UPDATE outbox SET sent=1 WHERE id=?`

- Si falla: log y dejar `sent=0` (reintenta automáticamente al siguiente tick — útil cuando la conexión cae transitoriamente)

HANDLER DE MENSAJES ENTRANTES

`messages.upsert` con `type: 'notify'` (ignorar 'append', 'replace').
Por cada mensaje:

1. Filtrar `msg.key.fromMe === true` (mensajes propios desde el teléfono del usuario).
2. Filtrar `remoteJid.endsWith('@g.us')` (grupos — fuera del scope v1).
3. Filtrar JID que no termine en `@s.whatsapp.net` (no es 1:1).
4. Extraer texto: `msg.message?.conversation` o `msg.message?.extendedTextMessage?.text`. Si no hay, ignorar (audio/imagen/sticker fuera del scope v1).
5. `getOrCreateConversation(phone, msg.pushName)`.
6. `insertMessage(convo.id, 'user', text)`.
7. RE-LEER conversation por id para chequear modo (toggle pudo haber pasado entre la creación y este check):
   ```typescript
   const fresh = getConversationById(convo.id);
   if (!fresh || fresh.mode !== 'AI') return;
   ```
8. Si modo AI: llamar al LLM con `getRecentHistory(20)` + system prompt, mapear roles `'human' → 'assistant'` para el LLM (los mensajes humanos del dashboard salieron del lado del bot, el LLM los ve como respuestas suyas previas), guardar reply como `role='assistant'`, enviar via `sock.sendMessage`.

Logging detallado — agrega logs `[bot] ← Mensaje de X: "..."`, `[bot] llamando LLM con N mensajes...`, `[bot] LLM respondió en Xms`, `[bot] → Enviado a Y`. Sirve mucho para debugging.

SYSTEM PROMPT INICIAL

```typescript
// src/lib/system-prompt.ts
export const SYSTEM_PROMPT = `
Eres un asistente virtual amable. Responde en español neutro,
en mensajes breves de 2 a 4 líneas. No uses emojis.
Si el usuario pide algo que no puedes resolver, responde:
"Déjame derivarte con un asesor humano."
`.trim();
```

El usuario va a personalizar este archivo después con el prompt de SU negocio. Documenta en el README cómo se hace.

DESCONEXIÓN MANUAL

`POST /api/connection/disconnect`:
1. `setConnectionState({status: 'disconnected', qr_string: null, phone: null})`
2. Borrar carpeta `./auth/` con `fs.rmSync(authDir, {recursive: true, force: true})`
3. Crear archivo flag `./data/.restart` (vacío)

`scripts/start-bot.ts` poll cada 1s `fs.existsSync('./data/.restart')`.
Si existe:
1. `fs.unlinkSync('./data/.restart')`
2. `await handle.shutdown()` (sock.logout() + sock.end())
3. `fs.rmSync('./auth/', {recursive: true, force: true})` (defensa)
4. Re-llamar `start()` para arrancar limpio (genera QR nuevo)

UI / TAILWIND

Paleta neutra (grises) + acento esmeralda para IA / ámbar para HUMANO.
Sin librerías de componentes (no shadcn, no Radix). Tailwind 4
vanilla.

Estados visuales clave:
- En `QRScreen`: `status === 'qr'` muestra "Esperando escaneo..." con
  punto ámbar pulsante. `'connecting'` muestra "Conectando..." azul.
  `'disconnected'` muestra spinner. Si lleva >10s en disconnected sin
  qr_string, mostrar mensaje de error con sugerencia de reiniciar
  el proceso bot.
- En `MessageBubble`: user a la izquierda (blanco con borde),
  assistant verde a la derecha, human ámbar a la derecha.
- En `ConversationList`: badge IA verde / HUMAN ámbar al lado de cada
  conversación. Mostrar "hace X min" como timestamp relativo.
- En `ConversationPanel`: input deshabilitado en modo IA (mostrar
  mensaje "El bot responde automáticamente"). Habilitado y enviable
  en modo HUMAN.

DEPLOY EN PRODUCCIÓN (EasyPanel sin Docker)

Si el usuario va a desplegar:

1. `Procfile` con `web: npm run start:all`
2. `nixpacks.toml`:
   ```toml
   providers = ["node"]
   [variables]
   NIXPACKS_NODE_VERSION = "22"
   [phases.setup]
   nixPkgs = ["nodejs_22", "npm-10_x", "python3", "gcc", "gnumake"]
   [phases.install]
   cmds = ["npm ci --include=dev"]
   ```

```
[phases.build]
cmds = ["npm run build"]
[start]
cmd = "npm run start:all"
```
3. `.nvmrc`: `22`
4. Volúmenes persistentes obligatorios: `/app/data` y `/app/auth`.
   Sin ellos cada redespliegue pierde conversaciones Y obliga a
   re-escanear el QR.

Documenta en el README: si los `:free` de OpenRouter no son
suficientes (50 req/día), recomienda `openai/gpt-4o-mini` ($0.15 por
millón de tokens — centavos por mes para uso normal).

SEGURIDAD — DASHBOARD SIN AUTH

El dashboard NO tiene autenticación. Documenta en el README:
si vas a desplegar a internet, ANTES pon basic auth (a nivel
proxy de EasyPanel/Caddy/Nginx) o Cloudflare Access. Si no, cualquiera
con la URL puede leer todas las conversaciones de WhatsApp y enviar
mensajes haciéndose pasar por el dueño. Marca esto como bloqueante
para producción.

REGLAS DE TRABAJO PARA TI (Claude Code)

1. Trabaja archivo por archivo en este orden:
   (a) `package.json` con dependencias, scripts, engines
   (b) `.env.example`, `.gitignore`, `tsconfig.json`, `next.config.ts`,
       `postcss.config.mjs`
   (c) `src/app/layout.tsx`, `src/app/globals.css`
   (d) `src/lib/db.ts` con DDL completo y todos los helpers
   (e) `src/lib/openrouter.ts` y `src/lib/system-prompt.ts`
   (f) `scripts/env-loader.ts` (separado, side-effect only)
   (g) `src/lib/baileys/client.ts` (con fetchLatestBaileysVersion,
       Browsers.macOS, state machine correcta)
   (h) `src/lib/baileys/handler.ts`
   (i) `scripts/start-bot.ts` (env-loader como PRIMER import)
   (j) API routes (status, disconnect, conversations, messages, mode,
       conversations/[id] DELETE)
   (k) Componentes en orden: ModeToggle, MessageBubble, ConversationList,
       ConversationPanel (con botón Borrar y composer HUMAN),
       DashboardHeader, QRScreen, ConnectionGate
   (l) `src/app/page.tsx`
   (m) `Procfile`, `nixpacks.toml`, `.nvmrc`
   (n) `README.md`

2. Después de los archivos críticos (db.ts, baileys/client.ts,
   handler.ts) muestra el código y espera confirmación. Para

boilerplate (configs, layout) puedes hacer batch sin preguntar.

3. Después de cada batch importante ejecuta `npx tsc --noEmit` para validar tipos. Si hay errores, arréglalos sin preguntar.

4. Cuando termines de declarar dependencias, ejecuta `npm install` tú mismo (avisa al usuario que tarda ~1 min por la compilación nativa de better-sqlite3).

5. NO inventes features fuera del scope. Si tienes ideas de mejora anótalas en una sección "Mejoras pendientes" del README.

6. NO uses Drizzle, Prisma, Supabase, Redis, WebSockets ni Vercel.

7. Idioma de comentarios en código y mensajes al usuario: español neutro.

8. Cuando el usuario quiera arrancar para probar:
   - Levanta el bot Y el dev server en background como tareas paralelas, muestra los logs principales.
   - Avisa que el QR aparece en localhost:3000.
   - Si el bot tira code=440 en loop:
     * Verifica que `Browsers.macOS('Desktop')` esté usado.
     * Pide al usuario que en su teléfono (Configuración → Dispositivos vinculados) borre cualquier dispositivo viejo de pruebas anteriores.
     * Si persiste, sugiere cambiar de IP del VPS o esperar 24h.
   - Si tira 429 en LLM: el modelo `:free` saturó la cuota. Recomienda `openai/gpt-4o-mini` en `OPENROUTER_MODEL`.

PRIMER PASO

Empieza mostrando el `package.json` que vas a generar (con `engines`, todas las deps en `dependencies` salvo las puramente de tipos) y espera confirmación antes de ejecutar `npm install`.

Empieza ahora.
```

---

## Notas para ti (el creador del curso)

Cosas que YO descubrí en el camino y que dejé incorporadas en el prompt para que tus suscriptores no las pisen:

1. **Code 405** — versión Baileys vs WhatsApp Web protocol. Fix: `fetchLatestBaileysVersion()` SIEMPRE.
```

2. **Code 440 en loop** — browser fingerprint custom. Fix: `Browsers.macOS('Desktop')`.

3. **Code 515** — es bueno, no malo. Es la señal de pairing exitoso. El bot solo necesita reconectar.

4. **QR no aparece en frontend** — el bot pasa por `qr → connecting` muy rápido, y la API solo miraba status estricto. Fix: API defensiva que muestra QR si `qr_string` existe.

5. **OPENROUTER_API_KEY undefined en bot** — ES module hoisting. Fix: env-loader como módulo separado importado primero.

6. **Procesos zombies** — TaskStop / Ctrl+C en Windows no siempre mata los hijos de tsx. Documentado para que sepan matarlos manualmente con tasklist + taskkill si ocurre.

7. **better-sqlite3 native build en Linux/Nixpacks** — pide python3, gcc, gnumake. Sin ellos el build remoto falla.

8. **Node 18 default en Nixpacks** — Baileys + Next 16 + Tailwind 4 requieren 20+. Fix: `engines.node` + `nixpacks.toml`.

9. **Modelos `:free` de OpenRouter** — 429 garantizado en producción real. Hay que recomendar pago desde el día uno.

10. **Dashboard sin auth** — riesgo crítico si se despliega. Marcado como bloqueante.

Si el suscriptor quiere expandir features:
- Soporte de imágenes salientes (enviar PNG de productos).
- Function calling real con `tools` de OpenRouter.
- Auto-toggle a HUMAN cuando el bot dice frase específica (detección por regex en `handler.ts`).
- WebSocket en lugar de polling.
- Auth básica en Next.js (middleware con basic auth).

Todo eso queda como "v2" en el README que arme Claude Code.